

Add_TwoNumbers :

```
import java.util.Scanner;

public class AddTwoNumbers{
    public static void main(String[] args){
        Scanner scanner=new Scanner(System.in);

        System.out.print("Enter First Number:");
        int num1=scanner.nextInt();

        System.out.print("Enter Second Number:");
        int num2=scanner.nextInt();

        int sum=num1+num2;
        System.out.println("The Sum is :"+sum);
    }
}
```

ATM :

```
public class ATM{
    public static void main (String[] args){
        double balance = 1000.0;
        double withdrawAmount = 1500.00;
        try{
            if(withdrawAmount>balance){
                throw new
Exception ("Insufficient Balance");
            }
            balance-=withdrawAmount;
            System.out.println("Withdrawal successfully Remaining balance:"+balance);
        }catch(Exception e){
            System.out.println("Error:"+e.getMessage());
        }
    }
}
```

Binary_insert :

```
class Binaryinsert {

    static class Node {
        int data;
        Node left, right;

        public Node(int item) {
            data = item;
            left = right = null;
        }
    }

    Node root;

    public Binaryinsert() {
        root = null;
    }
}
```



Edit with WPS Office

```

public Node insert(Node root, int key) {
    if (root == null) {
        root = new Node(key);
        return root;
    }
    if (key < root.data) {
        root.left = insert(root.left, key);
    } else if (key > root.data) {
        root.right = insert(root.right, key);
    }
    return root;
}

public Boolean search(Node root, int key) {
    if (root == null) {
        return false;
    }
    if (root.data == key) {
        return true;
    }
    if (key > root.data) {
        return search(root.right, key);
    }
    return search(root.left, key);
}

public void inorder(Node root) {
    if (root != null) {
        inorder(root.left);
        System.out.print(root.data + " ");
        inorder(root.right);
    }
}

public static void main(String[] args) {
    Binaryinsert tree = new Binaryinsert();

    tree.root = tree.insert(tree.root, 50);
    tree.root = tree.insert(tree.root, 30);
    tree.root = tree.insert(tree.root, 20);
    tree.root = tree.insert(tree.root, 40);
    tree.root = tree.insert(tree.root, 70);
    tree.root = tree.insert(tree.root, 60);
    tree.root = tree.insert(tree.root, 80);

    System.out.print("Inorder traversal of the BST: ");
    tree.inorder(tree.root);
    System.out.println();

    int keyToSearch = 40;
    System.out.println("Searching for node with key: " + keyToSearch);
    if (tree.search(tree.root, keyToSearch)) {
        System.out.println("Node with key " + keyToSearch + " found.");
    } else {
        System.out.println("Node with key " + keyToSearch + " not found.");
    }
}

```



```

        keyToSearch = 90;
        System.out.println("Searching for node with key: " + keyToSearch);
        if (tree.search(tree.root, keyToSearch)) {
            System.out.println("Node with key " + keyToSearch + " found.");
        } else {
            System.out.println("Node with key " + keyToSearch + " not found.");
        }
    }
}

```

BinarySearchTree:

```

class TreeNode1{
    int value;
    TreeNode1 left, right;

    TreeNode1(int value) {
        this.value = value;
        left = right = null;
    }
}

class BinarySearchTree {
    TreeNode1 root;

    TreeNode1 deleteNode(TreeNode1 root, int key) {
        if (root == null) {
            return root; // Tree is empty
        }

        if (key < root.value) {
            root.left = deleteNode(root.left, key);
        } else if (key > root.value) {
            root.right = deleteNode(root.right, key);
        } else {
            if (root.left == null) {
                return root.right;
            } else if (root.right == null) {
                return root.left;
            }

            root.value = minValue(root.right);
            root.right = deleteNode(root.right, root.value);
        }
        return root;
    }

    int minValue(TreeNode1 root) {
        int minValue = root.value;
        while (root.left != null) {
            root = root.left;
        }
        return root.value;
    }
}

```



```

TreeNode1 insert(TreeNode1 root, int value) {
    if (root == null) {
        root = new TreeNode1(value);
        return root;
    }

    if (value < root.value) {
        root.left = insert(root.left, value);
    } else if (value > root.value) {
        root.right = insert(root.right, value);
    }

    return root;
}

void inorderTraversal(TreeNode1 root) {
    if (root != null) {
        inorderTraversal(root.left);
        System.out.print(root.value + " ");
        inorderTraversal(root.right);
    }
}

public static void main(String[] args){
    BinarySearchTree bst = new BinarySearchTree();

    bst.root = bst.insert(bst.root, 50);
    bst.root = bst.insert(bst.root, 30);
    bst.root = bst.insert(bst.root, 70);
    bst.root = bst.insert(bst.root, 20);
    bst.root = bst.insert(bst.root, 40);
    bst.root = bst.insert(bst.root, 60);
    bst.root = bst.insert(bst.root, 80);

    System.out.println("In-order traversal of the BST:");
    bst.inorderTraversal(bst.root);
    System.out.println();

    int key = 80;
    System.out.println("Deleting node with value " + key);
    bst.root = bst.deleteNode(bst.root, key);

    System.out.println("In-order traversal after deletion:");
    bst.inorderTraversal(bst.root);
    System.out.println();
}
}

```

BinarySearchTree2

```

class Node {
    int key;
    Node left, right;

    public Node(int item) {
        key = item;
    }
}

```



Edit with WPS Office

```

        left = right = null;
    }
}

class BinarySearchTree2 {
    Node root;

    BinarySearchTree2() {
        root = null;
    }

    void insert(int key) {
        root = insertRec(root, key);
    }

    Node insertRec(Node root, int key) {
        if (root == null) {
            root = new Node(key);
            return root;
        }

        if (key < root.key)
            root.left = insertRec(root.left, key);
        else if (key > root.key)
            root.right = insertRec(root.right, key);

        return root;
    }

    void printTree() {
        printTree(root);
    }

    void printTree(Node node) {
        if (node != null) {
            printTree(node.left);
            System.out.print(node.key + " ");
            printTree(node.right);
        }
    }

    public static void main(String[] args) {
        BinarySearchTree2 tree = new BinarySearchTree2();

        tree.insert(50);
        tree.insert(30);
        tree.insert(20);
        tree.insert(40);
        tree.insert(70);
        tree.insert(60);
        tree.insert(80);

        System.out.println("BinarySearchTree2:");
        tree.printTree();
    }
}

```



Simple_Calculator:

```
import java.util.Scanner;
public class calculator{
    public static void main(String[] args){
        Scanner scanner=new Scanner(System.in);
        System.out.print("Enter first Number:");
        double num1=scanner.nextDouble();
        System.out.print("Enter second Number:");
        double num2=scanner.nextDouble();
        System.out.print("choose operation(+,-,*,/):");
        char operator=scanner.next().charAt(0);
        double result=0;
        switch (operator){
            case '+':
                result=num1+num2;
                System.out.println("The result is :"+ result);
                break;
            case '-':
                result=num1-num2;
                System.out.println("The result is :"+result);
                break;
            case '*':
                result=num1*num2;
                System.out.println("The result is :"+result);
                break;
            case '/':
                if(num2!=0){
                    result=num1/num2;
                }else{
                    System.out.println("Invalid ooperation:");
                    return;
                }
                System.out.println("The result is :"+result);
        }
    }
}
```

PrimeCheck :

```
import java.util.Scanner;
public class checkprime{
    public static void main(String[] args){
        Scanner scanner=new Scanner(System.in);

        System.out.print("Enter a number:");
        int num=scanner.nextInt();
        boolean isprime=true;

        if(num<=1){
            isprime=false;
        }else{
            for(int i=2; i<num/2; i++){
                if(num%i==0){
                    isprime=false;
                    break;
                }
            }
        }
    }
}
```



Edit with WPS Office

```

    }
}
}
if(isprime){
    System.out.println(num+ " is a prime number.");
}else{
    System.out.print(num+ " is not a prime number. ");
}
}
}
}

```

Even_Odd :

```

import java.util.Scanner;

public class evenodd{
    public static void main(String[] args){
        Scanner scanner=new Scanner(System.in);

        System.out.print("Enter a Number:");
        int num=scanner.nextInt();

        if(num%2==0){
            System.out.println(num + "is even");
        }else{
            System.out.println(num + "is odd.");
        }
    }
}

```

Factorial:

```

import java.util.Scanner;

public class factorial{
    public static void main (String[] args){
        Scanner scanner=new Scanner(System.in);

        System.out.print("Enter a number:");
        int num=scanner.nextInt();

        int factorial=1;
        for(int i=1;i<=num;i++){
            factorial*=i;
        }

        System.out.print("The Factorial of " +num+ " is"+factorial);
    }
}

```

Infix to Postfix :

```

import java.util.Stack;
public class InfixToPostfix{
    public static String convert(String infix){
        Stack<Character>stack = new Stack<>();
    }
}

```



Edit with WPS Office

```

        StringBuilder Postfix = new StringBuilder();
        for(char ch : infix.toCharArray()){
            if (Character.isLetterOrDigit(ch)) {
                Postfix.append(ch);
            }else if (ch=='('){
                stack.push(ch);
                Postfix.append(stack.pop());
            }
            stack.pop();
        }else{ if (ch==' '){
            while(!stack.isEmpty()&&precedence(stack.peek())>=precedence(ch)){
                Postfix.append(stack.pop());
            }
            stack.pop();
        }else{
        }
        while(!stack.isEmpty()){
            Postfix.append(stack.pop());
        }
        return Postfix.toString();
    }
    private static int precedence(char op){
        return (op == '+' || op == '-')?1:(op == '*' || op == '/')?2:0;
    }
    public static void main(String[] args){
        String infix="a-b*d/e(b+c)";
        System.out.println("Postfix:"+convert (infix ));
    }
}

```

Queue_ADT :

```

class Node {
    int data;
    Node next;
    public Node(int data){
        this.data=data;
        this.next=null;
    }
}
class Queue{
    Node front;
    Node rear;
    int size;
    public Queue(){
        front=rear=null;
        size=0;
    }
    public void enqueue(int data){
        Node newNode=new Node(data);
        if(rear==null){
            front=rear=newNode;
        }else{
            rear.next=newNode;
            rear=newNode;
        }
        size++;
    }
}

```




```

}
public int dequeue(){
    if(isEmpty()){
        throw new RuntimeException("Queue is empty");
    }
    int data=front.data;
    front=front.next;
    if(front==null)
    {
        rear=null;
    }
    size--;
    return data;
}
public Boolean isEmpty(){
    return front==null;
}
public int peek(){
    if(isEmpty()){
        throw new RuntimeException("Queue id Empty");
    }
    return front.data;
}
public int size(){
    return size;
}
}
public class Main{
    public static void main(String[] args){
        Queue queue=new Queue();
        queue.enqueue(10);
        queue.enqueue(20);
        queue.enqueue(30);
        System.out.println("Queue Size:"+queue.size());
        System.out.println("Front element:"+queue.peek());
        System.out.println("Dequeued element:"+queue.dequeue());
        System.out.println("Queue Size:"+queue.size());
    }
}

```

Postfix evaluation :

```

import java.util.Stack;
public class Postfixevaluation{
    public static int evaluatePostfix(String expression){
        Stack<Integer> stack=new Stack<>();
        for(int i=0;i<expression.length();i++){
            char ch=expression.charAt(i);
            if(Character.isDigit(ch)){
                stack.push(ch-'0');
            }else{
                int operand2=stack.pop();
                int operand1=stack.pop();
                switch(ch){
                    case '+':
                        stack.push(operand1 + operand2);

```



Edit with WPS Office

```

        break;
    case '-':
        stack.push(operand1 - operand2);
        break;
    case '*':
        stack.push(operand1 * operand2);
        break;
    case '/':
        stack.push(operand1 / operand2);
        break;
    }
}
}
return stack.pop();
}

public static void main(String[] args){
    String PostfixExpression="231*+9-";
    int result=evaluatePostfix(PostfixExpression);
    System.out.println("The result of the Postfix expression \"\" +PostfixExpression + "\" is:"
+result);
}
}

```

Primecheck :

```

import java.util.Scanner;
public class primecheck{
    public static void main(String[] args){
        Scanner scanner=new Scanner (System.in);

        System.out.print("Enter a Number:");
        int num = scanner.nextInt();
        boolean isprime=true;

        if(num<=1){
            isprime=false;
        }else{
            for(int i=2; i<num/2;i++){
                if(num%i==0){
                    isprime=false;
                    break;
                }
            }
        }

        if(isprime){
            System.out.println(num+ "is a prime number");
        }else{
            System.out.println(num+ "is not a prime number");
        }
    }
}
}
}

```

Reverse string :

```

import java.util.Scanner;

```



Edit with WPS Office

```

public class reversestring{
    public static void main(String[] args){
        Scanner scanner=new Scanner(System.in);

        System.out.print("Enter a number:");
        String input=scanner.next();

        String reversed="";
        for (int i=input.length()-1; i>=0; i--){
            reversed+= input.charAt(i);
        }
        System.out.println(" Reversed string " + reversed);
    }
}

```

Stack :

```

//Stack implementation in Java
class Stack{
    //Store element of Stack
    private int arr[];
    //represent top of Stack
    private int top;
    //total capacity of the Stack
    private int capacity;
    //Creating a Stack
    Stack(int Size){
        //initialize the array
        //initialize the Stack variables
        arr=new int[Size];
        capacity=Size;
        top=-1;
    }
    //push elements to the top of Stack
    public void push(int x){
        if(isFull()){
            System.out.println("StackOverFlow");
            //terminates the program
            System.exit(1);
        }
        //insert element on the top of Stack
        System.out.println("Inserting"+x);
        arr[++top]=x;
    }
    //pop elements from the top of Stack
    public int pop(){
        //if stack is empty
        //no element to pop
        if(isEmpty()){
            System.out.println("STACK EMPTY");
            //terminates the program
            System.exit(1);
        }
        //pop element from the top of Stack
        return arr[top--];
    }
}

```



```

    }
    //return size of the Stack
    public int getSize(){
        return top+1;
    }
    //check if the Stack is empty
    public Boolean isEmpty(){
        return top==--1;
    }
    //check if the Stack is full
    public Boolean isFull(){
        return top==capacity-1;
    }
    //display elements of Stack
    public void printStack(){
        for(int i=0; i<=top; i++){
            System.out.print(arr[i]+",");
        }
    }
    public static void main(String[] args){
        Stack stack=new Stack(5);
        stack.push(1);
        stack.push(2);
        stack.push(3);
        System.out.print("Stack:");
        stack.printStack();
        //remove element from stack
        stack.pop();
        System.out.println("\n After popping out");
        stack.printStack();
    }
}

```

Stack_adt:

```

class StackNode {
    int data;
    StackNode next;

    StackNode(int data) {
        this.data = data;
        this.next = null;
    }
}

class Stack {
    private StackNode top;

    public Stack() {
        this.top = null;
    }

    public void push(int data) {
        StackNode newNode = new StackNode(data);
        if (top == null) {
            top = newNode;
        }
    }
}

```



Edit with WPS Office

```

        } else {
            newNode.next = top;
            top = newNode;
        }
        System.out.println(data + " pushed to stack");
    }

    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack underflow");
            return -1;
        }
        int poppedData = top.data;
        top = top.next;
        return poppedData;
    }

    public int peek() {
        if (isEmpty()) {
            System.out.println("Stack is empty");
            return -1;
        }
        return top.data;
    }

    public boolean isEmpty() {
        return top == null;
    }

    public void display() {
        if (isEmpty()) {
            System.out.println("Stack is Empty");
            return;
        }
        StackNode current = top;
        System.out.println("Stack elements:");
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.next;
        }
        System.out.println();
    }
}

public class StackUsingLinkedList {
    public static void main(String[] args) {
        Stack stack = new Stack();
        stack.push(10);
        stack.push(20);
        stack.push(30);

        stack.display();

        System.out.println("Top element is: " + stack.peek());

        System.out.println(stack.pop() + " popped from stack");
    }
}

```



```

        stack.display();
    }
}

```

Tree node 1

```

class TreeNode1{
    int value;
    TreeNode1 left, right;

    TreeNode1(int value) {
        this.value = value;
        left = right = null;
    }
}

class BinarySearchTree {
    TreeNode1 root;

    // Method to delete a node from the BST
    TreeNode1 deleteNode(TreeNode1 root, int key) {
        if (root == null) {
            return root; // Tree is empty
        }

        // Recur down the tree
        if (key < root.value) {
            root.left = deleteNode(root.left, key);
        } else if (key > root.value) {
            root.right = deleteNode(root.right, key);
        } else { // Node to be deleted found
            // Case 1: Node has only one child or no child
            if (root.left == null) {
                return root.right;
            } else if (root.right == null) {
                return root.left;
            }

            // Case 2: Node has two children
            root.value = minValue(root.right); // Get in-order successor
            root.right = deleteNode(root.right, root.value); // Delete successor
        }
        return root;
    }

    // Helper method to find the minimum value in a subtree
    int minValue(TreeNode1 root) {
        int minValue = root.value;
        while (root.left != null) {
            root = root.left; // Find leftmost leaf
        }
        return root.value;
    }

    // Method to insert a new value into the BST
    TreeNode1 insert(TreeNode1 root, int value) {
        if (root == null) {

```



```

        root = new TreeNode1(value);
        return root; // If the tree is empty, create a new node
    }

    if (value < root.value) {
        root.left = insert(root.left, value); // Recur on left subtree
    } else if (value > root.value) {
        root.right = insert(root.right, value); // Recur on right subtree
    }

    return root; // Return the unchanged node pointer
}

// In-order traversal of the tree
void inorderTraversal(TreeNode1 root) {
    if (root != null) {
        inorderTraversal(root.left); // Recur on left subtree
        System.out.print(root.value + " "); // Print root value
        inorderTraversal(root.right); // Recur on right subtree
    }
}

public static void main(String[] args){
    BinarySearchTree bst = new BinarySearchTree(); // Create the BST

    // Insert values into the BST
    bst.root = bst.insert(bst.root, 50);
    bst.root = bst.insert(bst.root, 30);
    bst.root = bst.insert(bst.root, 70);
    bst.root = bst.insert(bst.root, 20);
    bst.root = bst.insert(bst.root, 40);
    bst.root = bst.insert(bst.root, 60);
    bst.root = bst.insert(bst.root, 80);

    // Display the BST before deletion
    System.out.println("In-order traversal of the BST:");
    bst.inorderTraversal(bst.root);
    System.out.println();

    // Delete node with value 50
    int key = 50;
    System.out.println("Deleting node with value " + key);
    bst.root = bst.deleteNode(bst.root, key);

    // Display the BST after deletion
    System.out.println("In-order traversal after deletion:");
    bst.inorderTraversal(bst.root);
    System.out.println();
}
}

```

